Adaptive Learning Rate Control Using Reinforcement Learning

Jack Peterson, Maxwell Woodfield, Mitchell Petingola, and Syed Mekael Wasti
School of Computing, Queen's University
Kingston, Canada
21jrp10@queensu.ca, 25pf15@queensu.ca, mitchell.petingola@queensu.ca, s.wasti@queensu.ca

***Abstract*—Static learning rate schedulers often struggle to navigate the complex loss landscapes of deep learning models, leading to suboptimal convergence or training instability. To address this, we propose a Reinforcement Learning (RL) framework that treats learning rate scheduling as an online sequential decision-making process. Specifically, we train an agent using Soft Actor-Critic (SAC) to dynamically adjust the learning rate based on observed signals from the convolutional neural network (CNN) training environments. Evaluations across different architectures and datasets demonstrate that this approach can achieve convergence that is similar to, and occasionally superior to, that of state-of-the-art static schedules. Additionally, we show that policies learned within specific training environments can be transferred to unseen architectures, delivering a promising alternative to manual hyperparameter tuning.**

***Index Terms*—learning rate, adaptive, control, reinforcement learning**

I. Introduction

The complexity of deep neural network loss landscapes, combined with the high computation cost of comprehensive experiments, makes identifying optimal hyperparameters challenging. Currently, grid search is a common method to tune hyperparameters, which involves a trial-and-error approach over a fixed range of values. However, to properly assess performance, a network needs to be trained multiple times to determine a "good' hyperparameter selection.

Among hyperparameters, the learning rate is arguably the most important for achieving effective model convergence. A large body of research has focused on developing robust learning rate schedules that reduce the need for exhaustive manual tuning. Fixed schedules such as step-decay, exponential decay, and cosine annealing can result in better convergence than static learning; however, they still require parameter tuning to determine an appropriate schedule.

Extending the work by Xu et al. [1], we use reinforcement learning to develop a closed-loop controller that dynamically adjusts the learning rate in response to real-time training metrics. We model the training environment as a partially observable Markov decision process (POMDP) and use Soft Actor-Critic, a more sample-efficient alternative to PPO, to learn an LR schedule.

We conduct several experiments to evaluate our framework's robustness and assess different observations, actions, and rewards. Additionally, we assess the feasibility of policy transferability, which measures the ability of a pre-trained RL agent to effectively control the learning rate for unseen architectures and datasets.

II. Related Work

Our methodology is based on the framework proposed by Xu et al. [1], which treats learning rate selection as a sequential decision-making problem. Their work demonstrates that an RL agent can dynamically control Adam learning rates to improve convergence and stability by reacting to real-time training statistics. We use this work as our foundation and extend its design and experiments.

Previous research has also explored the use of reinforcement learning for dynamic learning rate control. [2] demonstrated a similar approach on stochastic gradient descent (SGD) using actor-critic methods, and [3] adapted these schedulers specifically for convolutional neural networks (CNN). More recently, [4] explored a bandit-based approach for meta-RL learning rate control. In this work, the RL agent acts as a meta-optimizer, dynamically adjusting the learning rate of a separate, "trainee" RL agent.

Beyond learning rate control, recent work has explored using RL to dynamically adjust other hyperparameters. In particular, dynamic batch size control has shown potential [5], [6]. These works demonstrate that RL can effectively adjust batch size to account for gradient noise and hardware utilization.

III. Methodology

Our proposed agent architecture follows the methodology established by Xu et al. [1]. The agent operates as a learning rate (LR) controller that observes a state vector of training metrics from the environment every k optimizer steps. Based on these observations, the agent outputs an action value that scales the current LR, adaptively navigating the loss landscape. A high-level overview of the agent and environment is illustrated in Figure 1.

A. Soft Actor-Critic

While Xu et al. utilize Proximal Policy Optimization (PPO) to learn the controller policy, we propose Soft Actor-Critic (SAC) as an alternative [7]. SAC provides three main advantages over PPO: (1) its off-policy architecture uses a replay buffer to learn from past experience, significantly improving sample efficiency, (2) its maximum entropy objective prevents

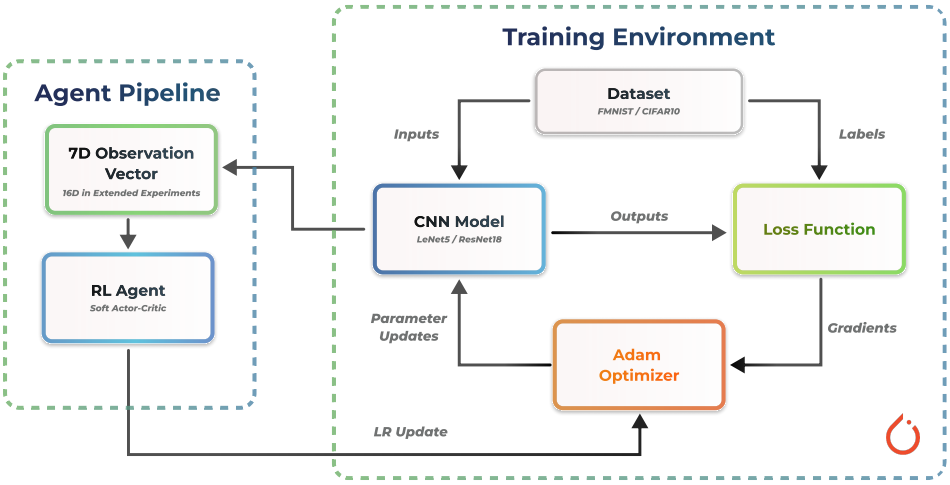


Fig. 1: Architecture of our CNN PyTorch Training Environment

premature convergence by improving exploration, and (3) it is inherently less sensitive to its own initial hyperparameters. Additionally, modern implementations of SAC dynamically adjust the temperature parameter α to maintain a target entropy. This eliminates the need to manually tune the exploration-exploitation trade-off, which is especially advantageous when the optimal value may differ between different training environments.

B. Observations

Although our ablation studies explore extended observation spaces, we use the formulation from [1] for our initial implementations. The agent observes a 7-D observation vector containing the following training environment signals:

1) Average training, L_{train}
2) Validation loss, L_{val}
3) Variance of predictions, σ^2_{pred}
4) Variance of prediction changes, $\sigma^2_{\Delta pred}$
5) Mean of the output weight matrix, μ_W
6) Variance of the output weight matrix, σ^2_W
7) Current learning rate, η

Formally, our raw observation vector at timestep t is

$$\tilde{\mathbf{s}}_t = \left[L_{train}, L_{val}, \sigma^2_{pred}, \sigma^2_{\Delta pred}, \mu_W, \sigma^2_W, \eta\right]$$

These signals provide the agent with a summary of the network's optimization progress, generalization, and stability.

Since our environment is modelled as a partially observable Markov decision process (POMDP) with noisy signals, we use an exponential moving average (EMA) to smooth the first 6 values of our observation vector,

$$\mathbf{s}_{t,1:6} = \alpha\tilde{\mathbf{s}}_{t,1:6} + (1-\alpha)\mathbf{s}_{t-1,1:6} \qquad \mathbf{s}_{t,7} = \tilde{\mathbf{s}}_{t,7}.$$

where the smoothing factor $\alpha \in (0, 1]$ determines how much weight is applied to recent observations. Since the current learning rate is deterministic, we leave it as a raw, unsmoothed value.

C. Actions

To provide the agent with fine-grained control and leverage the SAC algorithm's continuous policy optimization, we use a continuous action space for learning rate adjustments. At each environment step (i.e. every k optimizer steps), the agent outputs an action scale factor a_t that scales the current learning rate,

$$\eta_t = \text{clip}(a_t\eta_{t-1}, \eta_{min}, \eta_{max}) \qquad a_t \in \left[\frac{1}{a_{max}}, a_{max}\right]$$

where a_{max} determines the maximum scale factor allowed, and η_t is clipped to be within $[\eta_{min}, \eta_{max}]$ to prevent numerical stability issues. Compared to methods that output absolute values or apply incremental adjustments, this approach can generalize more effectively across different network architectures, as optimal learning rates often vary by orders of magnitude.

D. Reward

We replicate the reward function used by [1], where the reward r_t is defined as the negative validation loss at each environment step t,

$$r_t = -L_{val,t}.$$

Using the absolute validation loss, the agent is effectively tasked with minimizing the area under the validation loss curve. This reward prioritizes not only the speed of convergence, but also the final performance, as every step spent at a high loss value accumulates a larger penalty. The agent must learn to balance the trade-off between aggressive early-stage learning and the stability required to reach a lower global minimum.

IV. Experiments

To evaluate the performance and robustness of our methodology, we conduct several experiments across architectures and datasets and compare our agent against traditional baseline schedulers.

A. Experimental Setup

We implement our RL agent and training environments using the Stable Baselines framework [8], with the CNN training loop implemented in PyTorch [9]. Unless otherwise specified, we use the following hyperparameters for the trainee environment: an update frequency of $k = 10$, trainee network batch size of 1024, and 1000 optimizer steps per episode. The SAC agent uses actor and critic networks with two-layer MLPs of 32 and 64 hidden units, respectively. The agent is trained with a batch size of 256, a discount factor $\gamma = 0.99$, a learning rate $\alpha = 3 \times 10^{-4}$, and an automated entropy coefficient to maintain optimal exploration. We evaluate the learned policies over 10 epochs, recording the average and standard deviation of both loss and learning rate trajectories across five runs to account for stochastic variance.

B. Baseline Performance

The performance of our initial implementation is benchmarked against two baselines: a fixed learning rate and a step-decay schedule [10]. To ensure a fair comparison, we perform a grid search over the initial learning rate, step size, and decay factor. We evaluate on two models and dataset pairs: LeNet-5 on Fashion MNIST and ResNet-18 on CIFAR-10 [11]–[14]. These combinations cover small and medium-sized models, balancing computational efficiency and architecture complexity. While Xu et al. conducted similar experiments [1], we re-evaluate our own implementation to determine whether SAC can serve as an effective alternative to PPO. These experiments also serve as a baseline for further modifications to our observations, actions and rewards.

C. Ablation Studies

To extend the work of Xu et al. and provide a comprehensive evaluation of different observations, actions, and rewards, we conduct several ablation studies.

a) Extended Observation: We extend the baseline observation space, formulated by [1], from a 7-D observation vector to a 16-D space to enable the agent to consider optimization dynamics, temporal contexts, generalization-gap signals, and hyperparameter contexts:

8) Gradient normalization
9) Training loss change, ΔL_{train}
10) Validation loss change, ΔL_{val}
11) Epoch progress
12) Training accuracy
13) Validation accuracy
14) Weight decay
15) Batch size normalization
16) Dropout

b) Action Scale: We also perform an ablation study on the maximum action scale to determine the optimal trade-off between flexibility and stability. Because the agent modifies the learning rate using a multiplicative factor, a constrained scale ($a_{max} = 1.05$) ensures smoother updates but may affect the agent's ability to escape local minima efficiently. Conversely, a larger scale ($a_{max} = 2.0$) allows for more aggressive updates but can lead to poor stability if the agent overshoots the optimal learning rate. We learn a separate policy for each $a_{max} \in \{1.05, 1.25, 1.5, 2.0\}$ and evaluate them with LeNet-5 on Fashion MNIST.

c) Reward: One potential drawback of using the absolute validation loss as the reward is that the magnitude of the reward is non-stationary: as the model converges, the incentive for further improvements decreases. This change in magnitude can potentially lead to a weak reward signal during later stages of training. To address this, we investigate using a differential reward signal, calculated as the change in validation loss between environment steps,

$$r_t = L_{val,t-1} - L_{val,t}.$$

We hypothesize that this modification will help the agent identify what a "good" action is, since it will reward lower validation losses. For these experiments, we also use LeNet-5 on Fashion MNIST with $a_{max} = 1.25$ and an action scale.

D. Policy Transferability

Lastly, to assess the generalization capability of our agent, we conduct a policy transferability study. Specifically, we take the SAC policy trained on the ResNet-18/CIFAR-10 environment and deploy it, without any fine-tuning, to a ResNet-50 architecture. ResNet-50 has approximately twice the parameter count of ResNet-18 and uses "bottleneck" layers rather than standard basic blocks, leading to different gradient dynamics and loss landscapes. We compare the performance of this transferred policy against a static learning rate and a step-decay scheduler, both of which use hyperparameters chosen via grid search performed in the ResNet-50 environment. This experiment aims to determine if the agent has learned an architecture-agnostic policy for optimization, rather than overfitting to the specific dynamics of the training model.

V. Results

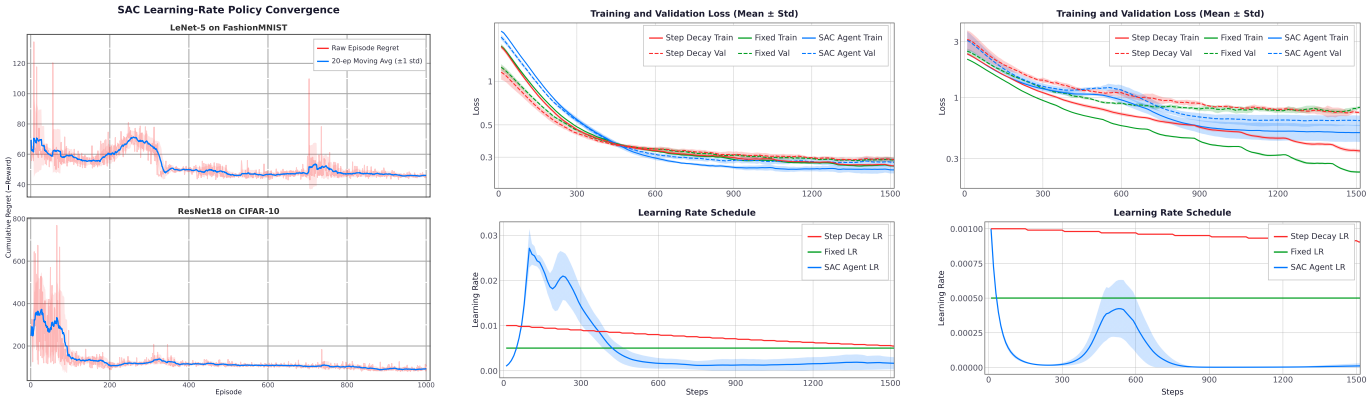
The policy convergence, loss curve, and LR schedule are illustrated in Figure 2, along with the final test loss and accuracy in Table II. For LeNet-5 on Fashion MNIST, all three schedulers demonstrated comparable performance, with no single schedule showing a statistically significant advantage over the baseline schedules. For ResNet-18 on CIFAR-10, there is a slight incremental improvement in test loss over the baseline schedules.

The results of the action scale ablation study are presented in Figure 3b with the final test loss and accuracy outlined in Table III. The results show that the more conservative action scale parameters achieve slightly better performance than more aggressive ones, with a final test accuracy of 89.28% for $a_{max} = 1.05$ and 89.57% for $a_{max} = 1.25$.

By extending the observation space from a 7D vector to a 16D vector, we provided the agent with significantly more training details, as shown in IV-C0a. The results in Table I demonstrate a clear relationship between the achieved accuracy of our trained SAC and the complexity of the underlying model and datasets. The SAC-16D variants consistently outperform 7D variants, with the performance gap

TABLE I: Test accuracy (%) across model/dataset combinations and observation space dimensions. Results reported as mean ± std over 3 runs. Best result per row is **bolded**.

		Fixed LR		Step Decay		Cosine		SAC (Ours)	
Model	**Dataset**	**7D**	**16D**	**7D**	**16D**	**7D**	**16D**	**7D**	**16D**
LeNet-5	FashionMNIST	87.53 ± 0.08	87.54 ± 0.22	89.69 ± 0.50	89.50 ± 0.43	86.41 ± 0.09	86.42 ± 0.16	88.93 ± 0.45	**89.81 ± 0.31**
	CIFAR-10	56.27 ± 0.72	56.33 ± 0.55	**58.41 ± 2.79**	58.31 ± 3.69	53.76 ± 1.24	53.71 ± 1.21	44.80 ± 3.55	52.44 ± 1.94
	CIFAR-100	22.99 ± 1.03	**23.18 ± 0.73**	19.35 ± 2.58	17.87 ± 4.74	20.17 ± 1.22	20.09 ± 1.31	14.78 ± 7.37	23.08 ± 4.02
ResNet-18	FashionMNIST	90.81 ± 0.35	90.34 ± 0.80	91.98 ± 0.75	89.63 ± 0.22	**93.65 ± 0.19**	93.84 ± 0.15	91.68 ± 1.00	90.12 ± 0.61
	CIFAR-10	76.96 ± 3.56	74.66 ± 4.36	77.04 ± 1.56	79.86 ± 2.35	**85.84 ± 0.19**	85.68 ± 0.29	68.66 ± 0.96	72.69 ± 2.43
	CIFAR-100	46.41 ± 2.61	46.55 ± 2.01	50.00 ± 1.91	44.30 ± 2.09	**59.64 ± 0.32**	59.60 ± 0.36	20.28 ± 1.12	26.85 ± 6.17



(a) LeNet-5 & ResNet-18 Convergence (b) LeNet-5 & Fashion MNIST Baseline (c) ResNet-18 & CIFAR10 Baseline

Fig. 2: Policy convergence and Agent vs. Baseline Learning Rate Scheduler evaluation results.

TABLE II: Comparative Performance

Dataset & Model	**Scheduler**	**Test Loss**	**Test Acc**
Fa. MNIST & LeNet-5	Fixed LR	0.298 ± 0.006	$89.2\% \pm 0.2\%$
	Step Decay	0.288 ± 0.011	$89.4\% \pm 0.4\%$
	SAC Agent	0.284 ± 0.016	$89.6\% \pm 0.7\%$
ResNet-18 & CIFAR-10	Fixed LR	0.736 ± 0.043	$78.0\% \pm 1.3\%$
	Step Decay	0.650 ± 0.064	$78.8\% \pm 1.9\%$
	SAC Agent	0.628 ± 0.065	$78.3\% \pm 2.3\%$

TABLE III: Action Scale on Performance (LeNet-5 on Fashion MNIST)

Action Scale	**Test Loss**	**Test Acc**
1.05	0.2950 ± 0.0047	$89.28\% \pm 0.37\%$
1.25	0.2904 ± 0.0048	$89.57\% \pm 0.32\%$
1.5	0.3278 ± 0.0103	$87.77\% \pm 0.44\%$
2.0	0.3390 ± 0.0103	$87.47\% \pm 0.44\%$

widening as task complexity increases. SAC-16D (89.81%) on LeNet-5/Fashion MNIST noticeably surpasses the best baseline scheduler, 7D step-decay. The impact of the expanded observation space is most evident on LetNet-5/CIFAR-100, where SAC-7D performance plummets to 14.78% while SAC-16D recovers up to 23.08%. Across ResNet-18 evaluations, all SAC configurations and alternative scheduling strategies underperform relative to cosine annealing. As explained in VII, many mini-ablations were attempted, but no clear solution prevailed.

The results of the policy transferability study are shown in Figure 4 and Table V. This experiment evaluates the performance of a SAC agent trained on ResNet-18 when applied to a larger ResNet-50 architecture on the CIFAR-10 dataset. The

TABLE IV: Alternate Reward Policy Performance (LeNet-5 on Fashion MNIST)

Scheduler	**Test Loss**	**Test Acc**
SAC Agent	0.3878 ± 0.0560	$85.84\% \pm 1.70\%$
Alternate Reward SAC Agent	0.0202 ± 0.0096	$87.82\% \pm 0.78\%$

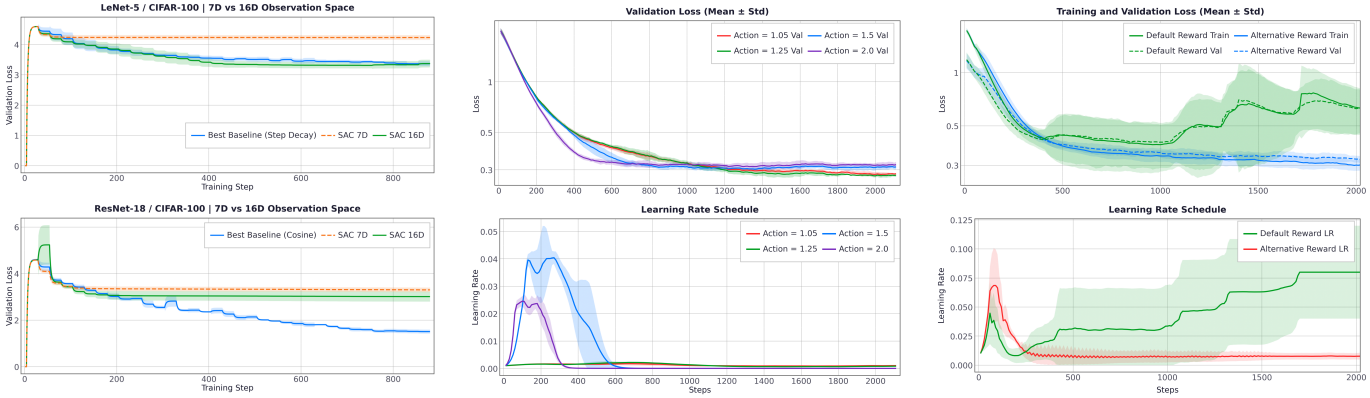
agent demonstrates strong generalization, outperforming both the fixed and step decay baseline schedules by achieving a final test accuracy of 78.89%, compared to 75.85% and 73.32%, respectively.

TABLE V: Policy Transferability Performance ResNet-18 → ResNet-50 on CIFAR-10

Scheduler	**Test Loss**	**Test Acc**
Fixed LR	0.7558 ± 0.0942	$75.85\% \pm 2.53\%$
Step Decay	0.8026 ± 0.1553	$73.32\% \pm 4.26\%$
SAC Agent (Exp. 8)	0.6138 ± 0.0522	$78.89\% \pm 1.98\%$

VI. Discussion

Overall, our SAC Agent performed similarly to step decay and fixed learning rate baselines. However, it did show an advantage in policy transferability performance on a larger and more complex architecture, suggesting that our method may be better suited to more complex training tasks. The biggest limitation of our methodology is the long training time, even for small models such as LeNet-5. As a result, transferability is necessary for any real-world usage on larger, more complex architectures. Our results do suggest that there is some potential for zero-shot application; however, this claim would need to be substantiated with more results.



(a) Observation Space (7D vs 16D) (b) Action Scale Factor (c) Alternative Reward Function

Fig. 3: Ablation studies comparing modified observation space, action scaling, and reward signals.

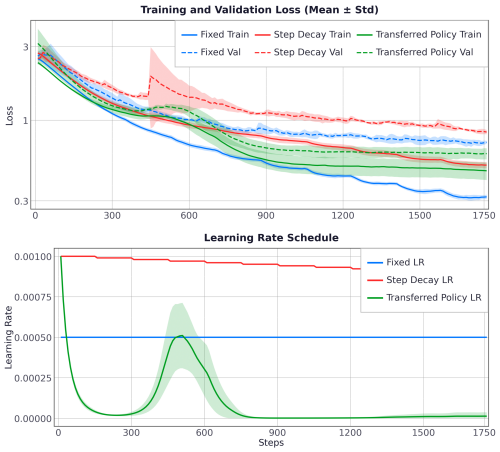


Fig. 4: Evaluation of policy transferability from ResNet-50 to CIFAR-10

A potential advantage of the RL agent is its ability to traverse around local minima by increasing the learning rate. In Figure 2b and Figure 2c, the SAC agent has two learning rate spikes mid-training. This spike occurring at a similar location across multiple runs suggests that there could be a local minimum that the agent dynamically avoids by scaling up the learning rate before lowering it again.

For our investigations into different action scale parameters, we found that an action scale factor of 1.25 was slightly more effective than 1.05 from the original paper's configuration. Increasing it any further resulted in more training instability and lower performance. Figure 3b illustrates that higher action scale factors do reach a smaller loss quicker than the lower values; however, they converge to a model with overall lower quality. Additionally, there is more variability in the final accuracy and loss per Table III. While smaller adjustments seem preferable to large adjustments, these results are likely affected by the initial learning rate, rather than the scaling factor being optimal, as the learning rate stays relatively consistent in both $a_{max} = 1.05$ and $a_{max} = 1.25$ policies. Overall, these results suggest that a smoother update with a lower action scale factor is preferable.

The results from our extended 16D observation-space experiments suggest that additional gradient and accuracy observations compensate for the noisier reward signals associated with complex models and datasets. While it was infeasible to perform per-feature ablation due to computational limitations, our results indicate that the extended observation space is not disrupting the agent's learning process. A deeper re-design of the reward strategies would be the reasonable next step, along with per-feature ablation experiments, which our codebase already supports.

We did find that the alternative reward we proposed did indeed outperform the original reward from Xu et al. Per Table IV, the alternative reward had a higher accuracy with lower variation compared to the original agent. This could be due to the potential for a positive reward, resulting in encouraging smarter actions. However, a side effect of this policy is that it creates a cyclic relationship in the learning rate. In Figure 3c, the alternative reward 'wobbles'; this could be due to the agent getting the highest reward when it takes a bad action and then following it with a good action, creating a large difference between the previous validation loss and the current validation loss. Consequently, our alternative reward function could be unintentionally rewarding bad actions. Overall, the benefit of rewarding good actions to a greater degree is positively altering the final policy. With more time, we would like to investigate a hybrid of the two rewards that balances rewarding good moves while ensuring that bad moves are not unintentionally rewarded.

One challenge we encountered with our agent is that the best action is almost always to lower the current learning rate, as a smaller learning rate will typically result in a lower validation loss without considering convergence time. In all of our figures, our agent usually immediately scales down to the lowest possible learning rate, while this may be an appropriate strategy in our CNN application, it may be better to instead reward convergence speed in addition to accuracy. Using validation loss as a reward fails to capture the subtleties of an ideal training scenario. Although expanding the agent's horizon to include previous validation loss slightly remedied

this by creating a cycle, the policy still primarily trends toward minimizing the learning rate. An interesting further experiment would be to use k previous validation losses as a reward or observation, as this could provide temporal relationships in the network training.

We did find that our SAC agent approach did adapt to a neural network architecture that it was not trained on. In Table V, our SAC agent outperformed the other approaches in accuracy by several percentage points. This higher accuracy may suggest that our SAC adjustment strategy can be used in zero-shot settings, which is a strong indication of its potential utility. If the agent can adapt to different unseen architectures, a potential utility is to offer several pre-trained learning rate controllers in a library. As the computational cost of applying a policy is low. Creating several pre-trained policies for different classes of training tasks would be an interesting direction for future work, as our results corroborate that policies can be applied to unseen architectures.

Overall, our work suggests that dynamically altering learning rates with reinforcement learning can improve CNN model performance, and we also found that by modifying the original MDP formulation by Xu et al [1], and using SAC as an alternative to PPO, we were able to increase accuracy and lower validation loss. Despite the potential of this methodology, the effects on accuracy and loss magnitude are very subtle and require substantially more policy training time compared to simply using an open-loop heuristic. A potential application would be to train several foundation policies for zero-shot training applications.

VII. Limitations and Future Work

Our study did have several limitations. Ideally, to demonstrate the robustness of our SAC agent, multiple policy training runs should be conducted to ensure the statistical validity of our work. Our SAC controller consistently underperformed the tuned step-decay baseline across model/dataset/observation combinations. We explored a range of interventions, a reshaped reward combining improvement and level terms, an extended 16D observation space (adding gradient norm, loss deltas, epoch progress, train/val accuracy, weight decay, batch size, and dropout), longer SAC training budgets (up to 100k agent steps), longer episodes (3000 trainee steps), a widened action scale ([1/1.5,1.5]), and a SAC learning-rate sweep, none of which bridged the gap. Within the scope of this project, we were unable to isolate a single root cause, despite significant time and compute spent on exploration. A significantly more time-intensive investigation (larger policy networks, more per-feature ablations, and paired statistical testing over more seeds) would be required to determine whether the gap reflects a fundamental limitation of the approach or a tractable implementation issue. We therefore report our results as a negative finding rather than a proof of concept. For future work, we would recommend investigating other policy algorithms, as SAC may not be the best fit for this problem. Additionally, performing studies on SOTA transformer architectures could provide insights into the practicality of our methodology for more modern problems. For future work, it would be interesting to investigate 'local minimum' detection from the observation space, as navigating around local minima would be a substantial advantage that reinforcement learning could have over open-loop heuristics.

VIII. Conclusion

In this work, we presented a Reinforcement Learning framework for adaptive learning rate control, formulating the problem as a sequential decision-making process and using a Soft Actor-Critic to dynamically learn policies. Our results show that the proposed method achieves performance comparable to, and in some instances exceeding, traditional learning rates, specifically fixed or step-decay schedulers across multiple architectures and datasets.

Acknowledgments

AI Disclosure: Generative AI was used during implementation to provide boilerplate environment templates, debug code, and assist with plot generation. All generated code was carefully reviewed for correctness. For paper preparation, AI was only used for refining grammar and clarity.

References

[1] Z. Xu, A. M. Dai, J. Kemp, and L. Metz, "Learning an adaptive learning rate schedule." [Online]. Available: https://arxiv.org/abs/1909.09712

[2] C. Xu, T. Qin, G. Wang, and T.-Y. Liu, "Reinforcement learning for learning rate control." [Online]. Available: https://arxiv.org/abs/1705.11159

[3] L. Wen, X. Li, and L. Gao, "A new reinforcement learning based learning rate scheduler for convolutional neural network in fault classification," vol. 68, no. 12, pp. 12 890–12 900.

[4] H. Donâncio, A. Barrier, L. F. South, and F. Forbes, "Dynamic learning rate for deep reinforcement learning: A bandit approach." [Online]. Available: https://arxiv.org/abs/2410.12598

[5] Y. Dai, K. He, and A. Wang, "Dynamix: Rl-based adaptive batch size optimization in distributed machine learning systems," 2025. [Online]. Available: https://arxiv.org/abs/2510.08522

[6] A. Devarakonda, M. Naumov, and M. Garland. AdaBatch: Adaptive Batch Sizes for Training Deep Neural Networks. [Online]. Available: http://arxiv.org/abs/1712.02029

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor." [Online]. Available: https://arxiv.org/abs/1801.01290

[8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.

[10] R. Ge, S. M. Kakade, R. Kidambi, and P. Netrapalli, "The step decay schedule: A near optimal, geometrically decaying learning rate procedure for least squares." [Online]. Available: https://arxiv.org/abs/1904.12838

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," vol. 86, no. 11, pp. 2278–2324.

[12] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms." [Online]. Available: https://arxiv.org/abs/1708.07747

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition." [Online]. Available: https://arxiv.org/abs/1512.03385

[14] F. O. Giuste and J. C. Vizcarra, "CIFAR-10 image classification using feature ensembles." [Online]. Available: https://arxiv.org/abs/2002.03846